



Security Assessment Report



Veda Swapper

May 2026

Prepared for Veda

Table of Contents

Project Summary	4
Project Scope	4
Project Overview	4
Protocol Overview	5
Assessment Methodology	6
Threat and Security Overview	7
Findings Summary	10
Severity Matrix	10
Detailed Findings	11
High Severity Issues	14
H-01: Malicious Strategist can steal funds from a swapper due to missing `_validateAdapter` in `_cancelOrder` function	14
H-02: Malicious pool may bypass token validation and drain the swapper across multiple adapter functions	16
Medium Severity Issues	23
M-01: Strategist can submit a partial-fill 1inch limit order, breaking the protocol's full-fill invariant	23
M-02: `FeeRegistry.getFee()` returns the swapper default fee for pairs that were explicitly configured to zero	25
M-03: Strategist can refund the protocol fee on an executed limit order to the vault via miscalculated `availablePrincipal` in `_cancelOrder()`	27
M-04: Lack of check on filled orders allows premature fee release and already executed order cancellation	30
M-05: Instant swaps overwrite and clear token allowances of pending limit orders	33
M-06: Slippage check in atomic swaps should be done with amounts after fees	35
M-07: `GenericRateProviderWithStalenessCheck` stores decimal metadata but does not normalize the rate	38
Low Severity Issues	41
L-01: Protocol fee should round up, in favor of the protocol	41
L-02: Slippage threshold rounds down, letting swaps with slightly worse output pass	42
L-03: Lowering `maxFeeBps` does not retroactively cap existing fee configurations	43
L-04: `verifyLimitOrder` does not validate extension-to-salt binding, relying on 1inch protocol-level check	46
L-05: Rebasing tokens can break swap and order accounting	48
L-06: A solver can tamper with `quoteAsset` and `slippageBps` at fill time to bypass the price check in `isValidSignature()`	50
L-07: Wrong path validation in `UniswapV3Adapter.exactInput` allows a malicious strategist to steal from the swapper	52
Informational Severity Issues	56
I-01: Failed swap consumes additional gas before reverting	56

I-02: Refactor and document <code>BoringSwapper::replaceOrder()</code>	57
I-03: <code>getFeeRecipient()</code> natspec describes a fee, not a recipient.....	58
I-04: Unsanitized <code>swapData</code> selector allows limit-order adapter functions to be dispatched through the atomic swap path, producing misinterpreted return values.....	59
I-05: Missing upper-bound validation on basis-point values allows fees above 100%, which would cause DoS on atomic swaps.....	61
I-06: Duplicate <code>protocolHash</code> is accepted.....	63
I-07: <code>cancelOrder</code> / <code>replaceOrder</code> lack <code>nonReentrant</code>	64
I-08: <code>OneInchAdapter</code> does not block the WETH-unwrap flag on <code>unoswap*</code> paths and <code>fillOrder</code>	65
I-09: Malicious strategist can route vault funds to arbitrary contracts via uninspected inner call targets in LI.FI and OpenOcean adapters.....	68
Disclaimer	71
About Certora	71

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Veda Swapper	https://github.com/Veda-Labs/boring-vault/tree/feat/swapper	5c768d3 (initial) fb7d2bb5 (fix)	Solidity

Project Overview

This document describes the security review of **Veda Swapper**. The work was undertaken from **May 6th, 2026** to **May 18th, 2026**.

The following contract list is included in our scope:

```
src/base/Periphery/BoringSwapper.sol
src/base/Periphery/AdapterRegistry.sol
src/base/Periphery/FeeRegistry.sol
src/base/Periphery/adapters/price/PriceValidator.sol
src/base/DecodersAndSanitizers/Protocols/BoringSwapperDecoderAndSanitizer.sol
src/base/Periphery/adapters/BaseAdapter.sol
src/base/Periphery/adapters/CowswapAdapter.sol
src/base/Periphery/adapters/UniswapV3Adapter.sol
src/base/Periphery/adapters/OneInchAdapter.sol
src/base/Periphery/adapters/OpenOceanAdapter.sol
src/base/Periphery/adapters/LifiAdapter.sol
src/helper/GenericRateProviderWithStalenessCheck.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The BoringSwapper protocol is a swap execution layer for BoringVault instances, designed to let authorized strategists trade vault assets through external DEX aggregators and limit order protocols while enforcing oracle-based slippage protection and protocol fee collection.

It centers around a single BoringSwapper contract that supports two execution modes: atomic swaps, where tokenIn is pulled from the vault, routed through an approved adapter in a single transaction, and tokenOut is returned immediately, and limit orders, where the swapper locks principal on behalf of the vault and registers an EIP-1271 on-chain authorization that external settlement protocols can fill asynchronously.

Each swap call is routed through ManagerWithMerkleVerification, which enforces Merkle-verified allowlists on token pairs and receiver addresses before forwarding to the swapper, forming the primary on-chain access control layer for strategist calls. Protocol-specific encoding and validation is delegated to stateless adapter contracts — OnInchAdapter, LifiAdapter, OpenOceanAdapter, CowswapAdapter, and UniswapV3Adapter — each of which decodes calldata, verifies that token routes and receivers match the approved configuration, and returns the swap target and input amount back to the swapper. A PriceValidator contract performs oracle-based slippage checks using configurable IRateProvider sources, supporting both direct and intermediary-hop pricing.

Fees are managed by an external FeeRegistry controlled exclusively by Veda: for atomic swaps, a percentage of tokenOut is withheld and transferred to a fee recipient at settlement time; for limit orders, a fee in tokenIn is escrowed alongside the principal at submission and released to Veda after off-chain confirmation of a successful fill or returned to the vault after a configurable delay on cancellation.

A global AdapterRegistry maintained by Veda tracks which adapter contracts are legitimate, and each swapper instance maintains its own per-adapter approval on top of the global registry, requiring both to be satisfied before any adapter can be used.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The BoringSwapper is a modular swap execution engine designed to serve one or more BoringVaults. It holds assets transiently — only during the window between fund receipt and execution — and relies on a set of governance-approved adapters to translate high-level SwapConfig objects into concrete calldata for external DeFi protocols. Two execution modes are supported: atomic swaps, where the swapper acts as taker against on-chain liquidity in a single transaction, and limit orders, where the swapper acts as maker, places a resting order on an off-chain orderbook such as CoW Protocol, 1inch, or OpenOcean, and waits for an external solver to fill it. Supporting infrastructure includes a FeeRegistry mapping swapper and token-pair tuples to fee rates and recipients, a PriceValidator enforcing oracle-backed slippage bounds on every swap, and a family of IRateProvider implementations supplying live price data from Chainlink or arbitrary on-chain callables.

The core invariants the audit set out to verify are: no token flow out of the swapper bypasses oracle-validated slippage; an adapter may only be invoked if it appears on the on-chain allowlist and the swapper is unpaused; every limit order's output token route is locked to the vault at submission time and remains locked through fill; the swapper's balance for any denomination satisfies at all times the floor defined by pendingOrderPrincipal plus feesInToken plus claimableFees; and limit orders are fully filled exactly once. Trust is tiered: governance sets the adapter allowlist, oracle configurations, fee parameters, and rate limits; strategists are trusted to select swap paths and order parameters within those boundaries but are explicitly not trusted to be honest; and solvers and off-chain bots interact only through isValidSignature, which is treated as a re-entry point that must independently revalidate every invariant established at order submission.

The audit focused on five broad attack surfaces. The first was the adapter dispatch surface: every code path invoking an adapter was checked for the four guards — pause state, allowlist membership, token-route binding, and receiver binding. The cancel path was found to be missing the allowlist guard entirely, allowing a malicious strategist to substitute a crafted adapter that returns arbitrary calldata executed with the swapper as msg.sender against a protocol already holding a live ERC-20 approval, enabling full drainage of approved balances. The second surface was limit order field completeness: each adapter's verifyLimitOrder was compared against the

full field set of the underlying protocol's order struct. The linch adapter omits makerTraits flag checks, permitting partial or multi-fill orders that violate the single-full-fill invariant and corrupt the bookkeeping counters. The third surface was the EIP-1271 callback. The isValidSignature flow was audited for solver-supplied inputs that bypass the immutable on-chain state. The quoteAsset and slippageBps fields were found to pass from the solver payload directly into PriceValidator.validate without being bound to the stored OrderRecord. A solver can substitute a quoteAsset whose oracle configuration carries skipValidation equal to true, causing the validator to return immediately and accept any fill price. The fourth surface was the fee bookkeeping model. The three counters were traced through every state transition: _cancelOrder derives availablePrincipal from a subtraction that accounts only for the current order's own fee and misses the aggregate locked floor, enabling redirection of a filled order's protocol fee to the vault; and releaseFee performs no on-chain fill check, permitting premature fee extraction before any settlement occurs. The fifth surface covered the approval lifecycle, oracle correctness, slippage timing, and path encoding. Instant swaps were found to zero the approval for the same target and token used by a co-existing limit order, silently revoking the resting order's allowance. GenericRateProviderWithStalenessCheck stores inputDecimals and outputDecimals but does not apply them, returning raw oracle values that break the 18-decimal assumption baked into PriceValidator. The oracle validation in _swapPostFlightCheck operates on the gross tokenBalanceDelta before fee deduction, widening the vault's effective slippage by feeBps. The UniswapV3Adapter reads the first and last 20 bytes of a path without validating the canonical 23-bytes-per-hop structure, allowing a crafted path whose terminal bytes spell out the expected tokenOut while Uniswap executes an intermediate hop to a different token. Several adapter functions in OneInchAdapter and OpenOceanAdapter validate the output token solely by calling tokenO() and tokenI() on caller-supplied pool addresses with no factory registry check, allowing a malicious strategist to supply a contract that absorbs tokenIn without delivering tokenOut.

Manual review covered all functions in BoringSwapper, FeeRegistry, PriceValidator, OneInchAdapter, OpenOceanAdapter, UniswapV3Adapter, and CowswapAdapter, as well as the GenericRateProviderWithStalenessCheck contract. AggregationRouterV6 was read to trace the uniswap, fillOrder, and _fill internals relevant to adapter trust assumptions. The CoW settlement contract and linch Limit Order Protocol were cross-referenced to verify field semantics, fill-state storage, and fill-check APIs. Integration tests were executed against a mainnet fork to exercise the complete atomic swap path using real on-chain liquidity and to produce a working proof of concept for the cancel-drain vector. The isValidSignature slippage bypass was verified

analytically. Formal verification of the bookkeeping arithmetic and fuzz testing of the fee rounding edge case were not performed and represent the primary coverage gaps.

Two high-severity issues dominate the risk profile. The missing adapter allowlist guard in `_cancelOrder` collapses the swapper's entire adapter-isolation model for cancel and replace operations; its blast radius is bounded only by existing ERC-20 approvals on the cancel target, which for 1inch and OpenOcean is the same contract that holds the fill approval. The pool address validation gap in the 1inch and OpenOcean direct-path functions allows a strategist to supply a contract returning plausible `tokenO/tokenI` while absorbing `tokenIn`, with the post-execution oracle check as the only backstop. At the medium tier, the approval lifecycle issue and the bookkeeping race in `_cancelOrder` are operationally coupled: the former can cause a legitimate limit order to fail silently while the latter enables exploitation of the resulting fill ambiguity to redirect protocol fees. Together they represent a meaningful fragility in the concurrent atomic-swap and limit-order modes that requires both code fixes and sequencing constraints in the off-chain bot logic.

The BoringSwapper integrates with five external protocols — 1inch (both the AggregationRouterV6 and the Limit Order Protocol), OpenOcean, CoW Protocol, UniswapV3 and LiFi — each of which exposes a broad and complex execution surface of its own. A thorough assessment of every interaction boundary between the in-scope contracts and these external systems would require dedicated time beyond what was available for this engagement. The team prioritized the most critical interaction points, cross-referenced the relevant contract internals to the fullest extent possible within the engagement window, and is confident that the most impactful integration risks have been identified and documented. We recommend that any remaining integration surfaces be reviewed as part of a follow-on assessment prior to production deployment.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	2	2	2
Medium	7	6	5
Low	7	7	5
Informational	9	9	8
Total	25	25	20

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
H-01	Malicious Strategist can steal funds from a swapper due to missing `_validateAdapter` in `_cancelOrder` function	High	Fixed
H-02	Malicious pool may bypass token validation and drain the swapper across multiple adapter functions	High	Fixed
M-01	Strategist can submit a partial-fill 1inch limit order, breaking the protocol's full-fill invariant	Medium	Acknowledged
M-02	`FeeRegistry.getFee()` returns the swapper default fee for pairs that were explicitly configured to zero	Medium	Fixed
M-03	Strategist can refund the protocol fee on an executed limit order to the vault via miscalculated `availablePrincipal` in `_cancelOrder()`	Medium	Fixed
M-04	Lack of check on filled orders allows premature fee release and already executed order cancellation	Medium	Fixed
M-05	Instant swaps overwrite and clear token allowances of pending limit orders	Medium	Fixed

M-06	Slippage check in atomic swaps should be done with amounts after fees	Medium	Acknowledged
M-07	<code>`GenericRateProviderWithStalenessCheck`</code> stores decimal metadata but does not normalize the rate	Medium	Fixed
L-01	Protocol fee should round up, in favor of the protocol	Low	Fixed
L-02	Slippage threshold rounds down, letting swaps with slightly worse output pass	Low	Fixed
L-03	Lowering <code>`maxFeeBps`</code> does not retroactively cap existing fee configurations	Low	Fixed
L-04	<code>`verifyLimitOrder`</code> does not validate extension-to-salt binding, relying on linch protocol-level check	Low	Acknowledged
L-05	Rebasing tokens can break swap and order accounting	Low	Acknowledged
L-06	A solver can tamper with <code>`quoteAsset`</code> and <code>`slippageBps`</code> at fill time to bypass the price check in <code>`isValidSignature()`</code>	Low	Fixed
L-07	Wrong path validation in <code>`UniswapV3Adapter.exactInput`</code> allows a malicious strategist to steal from the swapper	Low	Fixed

I-01	Failed swap consumes additional gas before reverting	Informational	Fixed
I-02	Refactor and document <code>`BoringSwapper::replaceOrder()`</code>	Informational	Fixed
I-03	<code>`getFeeRecipient()`</code> natspec describes a fee, not a recipient	Informational	Fixed
I-04	Unsanitized <code>`swapData`</code> selector allows limit-order adapter functions to be dispatched through the atomic swap path, producing misinterpreted return values	Informational	Fixed
I-05	Missing upper-bound validation on basis-point values allows fees above 100%, which would cause DoS on atomic swaps	Informational	Fixed
I-06	Duplicate <code>`protocolHash`</code> is accepted	Informational	Fixed
I-07	<code>`cancelOrder`</code> / <code>`replaceOrder`</code> lack <code>`nonReentrant`</code>	Informational	Fixed
I-08	<code>`OneInchAdapter`</code> does not block the WETH-unwrap flag on <code>`unoswap*`</code> paths and <code>`fillOrder`</code>	Informational	Fixed
I-09	Malicious strategist can route vault funds to arbitrary contracts via uninspected inner call targets in LI.FI and OpenOcean adapters	Informational	Acknowledged

High Severity Issues

H-01: Malicious Strategist can steal funds from a swapper due to missing `\_validateAdapter` in `\_cancelOrder` function

Severity: High	Impact: High	Likelihood: Low
Files: /src/base/Periphery/BoringSwapper.sol	Status: Fixed	

Description:

Every entry point in the `BoringSwapper` contract that invokes an adapter calls `_validateAdapter` first. All except `_cancelOrder` (called by both `cancelOrder` and `replaceOrder`). The function goes directly to:

```
address adapter = swapConfig.adapter;    // caller-supplied, never validated
IAAdapter.OrderInfo memory info = IAAdapter(adapter).verifyLimitOrder(swapConfig,
address(this));
if (info.protocolHash != record.protocolHash) revert
BoringSwapper__CancelFailed();
```

The lack of adapter validation would allow a malicious Strategist to do several bad things on the protocol, but mainly it would allow them to drain funds from the swapper.

A rogue Strategist can craft a malicious adapter contract that bypasses the rest of `_cancelOrder`'s safeguards. First, the stored `protocolHash` is readable on-chain via `orderRecords[orderId].protocolHash`, or reconstructed from the `submitOrder` transaction calldata / `OrderSubmitted` event. A malicious adapter simply returns that value from `verifyLimitOrder`, passing the check unconditionally.

After the hash check, the function calls the adapter's `cancelLimitOrder` and, if it returns a non-empty `data` field, calls the `record.cancelTarget` contract with said `data`:

```
(address target, bytes memory data) =
IAAdapter(adapter).cancelLimitOrder(swapConfig, address(this));
```

```
if (data.length > 0) {
    if (target != record.cancelTarget) revert BoringSwapper__CancelFailed();
    (bool success,) = target.call(data);    // arbitrary calldata, msg.sender =
    swapper
    ...
}
```

For **linch** and **OpenOcean** orders, **cancelTarget == approvalTarget**, the same contract that already holds a live ERC20 approval from the swapper (granted at **submitOrder**). Any function on that contract is reachable with the swapper as **msg.sender**, enabling an attacker to drain all the allowance the **approvalTarget** contract has of any token from the swapper by crafting calldata that triggers a token pull through the existing approval (e.g. executing a swap on the protocol on behalf of the swapper with the receiver being an attacker-controlled address).

Recommendations:

Add **_validateAdapter(swapConfig.adapter)** at the top of **_cancelOrder**, mirroring every other adapter-dispatching code path:

```
function _cancelOrder(uint256 orderId, SwapConfig calldata swapConfig) internal {
    _validateAdapter(swapConfig.adapter);    // add this line
    OrderRecord storage record = orderRecords[orderId];
    ...
}
```

This restores all four guard checks (pause state, registry membership, local approval) and closes all possible attack paths caused by the arbitrary execution described in the report.

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed, but we noticed now orders can be cancelled even if the protocol is paused. It's probably not a problem but we wanted to raise this.

Customer Response:

Yes this is intentional.

H-02: Malicious pool may bypass token validation and drain the swapper across multiple adapter functions

Severity: High	Impact: High	Likelihood: Medium
Files: /src/base/Periphery/adapters/OnInchAdapter.sol /src/base/Periphery/adapters/OpenOceanAdapter.sol	Status: Fixed	

Description:

Several adapter functions validate the output token solely by calling `token0()` and `token1()` on caller-supplied pool addresses, with no check that those addresses belong to any legitimate factory. Any contract that returns the expected token addresses from those view functions passes the adapter's guard while its execution logic routes `tokenIn` to the attacker. This affects `OnInchAdapter` and `OpenOceanAdapter`.

`OnInchAdapter`: `unoswap`, `unoswap2`, `unoswap3`, `unoswapTo`, `unoswapTo2`, `unoswapTo3`

`OnInchAdapter` resolves `tokenOut` by calling `token0()` and `token1()` on the pool address embedded in the caller-supplied `dex` uint256:

```
function _getTokenOutUni(uint256 dex, address tokenIn) internal view returns (address) {
    address pool = address(uint160(dex));
    return IUniswapV3(pool).token0() == tokenIn ? IUniswapV3(pool).token1() :
    IUniswapV3(pool).token0();
}
```

The pool address comes entirely from bits 0–159 of `dex`. There is no factory check. Any malicious contract returning the expected `tokenOut` (either for Uni V2/V3 or for Curve) will pass validation:

```
function unoswap(
    uint256 token,
    uint256 amount,
    uint256,
```



```
    /*minReturn*/
    uint256 dex
)
    external
    view
    returns (address, uint256)
{
    BoringSwapper.SwapConfig memory swapConfig = _getAppendedSwapConfig();
    if (ERC20(address(uint160(token))) != swapConfig.tokenRoute.tokenIn)
revert OneInchAdapter__TokenInMismatch();

    address tokenOut = _unoswapCheck(dex, token);

    if (ERC20(tokenOut) != swapConfig.tokenRoute.tokenOut) revert
OneInchAdapter__TokenOutMismatch();
    return (router, amount);
}
...

function _unoswapCheck(uint256 dex, uint256 token) internal view returns
(address) {
    return _getTokenOut(dex, address(uint160(token)));
}
...

function _getTokenOut(uint256 dex, address tokenIn) internal view returns
(address) {
    uint8 protocol = _protocol(dex);
    if (protocol == 0 || protocol == 1) return _getTokenOutUni(dex, tokenIn);
    if (protocol == 2) return _getTokenOutCurve(dex);
    revert OneInchAdapter__UnsupportedProtocol();
}
...

function _getTokenOutCurve(uint256 dex) internal view returns (address) {
    //extract the pool address
    address pool = address(uint160(dex));
    uint256 toTokenIndex = (dex >> CURVE_TO_COINS_ARG_OFFSET) &
CURVE_TO_COINS_ARG_MASK;
```

```
    return ICurvePool(pool).coins(toTokenIndex);  
}
```

The `unoswapTo`, `unoswapTo2`, and `unoswapTo3` variants are equally vulnerable: they add an explicit `to` address that is validated to equal the swapper, but the pool addresses (`dex`, `dex2`, `dex3`) go through the same `_unoswapCheck`/`_unoswap2Check`/`_unoswap3Check` helpers with no factory validation.

Once the adapter returns `(router, amount)`, the BoringSwapper calls the 1inch router with the original calldata. The 1inch `AggregationRouterV6` does not validate pool legitimacy either, so a malicious pool may end up keeping all the `tokenIn` amount sent in exchange of zero `tokenOut`.

Exception: single-hop UniV3 (`unoswap` / `unoswapTo` with a UniV3 `dex`): when BoringSwapper is the direct token payer, `_unoswapV3` encodes `spender = msg.sender` in the callback data. The router's `uniswapV3SwapCallback` then verifies the caller via `CREATE2` (`_FF_FACTORY`, `_POOL_INIT_CODE_HASH`), rejecting any pool not deployed by the canonical Uniswap V3 factory, the same protection OpenOcean uses (see below). A malicious pool cannot complete the swap in this configuration. The vulnerability therefore applies to: single-hop UniV2 and Curve; and any second- or third-hop pool (`dex2`, `dex3`) in two- and three-hop routes, regardless of protocol.

OpenOceanAdapter: `callUniswap`, `callUniswapTo`

`OpenOceanAdapter` resolves `tokenOut` by calling `token0()` or `token1()` on the last element of the caller-supplied `pools` array:

```
function _getUniV2DstToken(bytes32[] calldata pools) internal view returns  
(address) {  
    uint256 rawPool = uint256(pools[pools.length - 1]);  
    if (rawPool & WETH_MASK != 0) revert  
OpenOceanAdapter__WethFlagsNotAllowed();  
    address poolAddr = address(uint160(rawPool));  
    bool reversed = rawPool & REVERSE_MASK != 0;  
    return reversed ? IUniswapV3(poolAddr).token0() :  
IUniswapV3(poolAddr).token1();  
}
```

Again, there is no factory check. The OpenOcean UniV2 router (`UniswapV2Exchange._callUniswap`) also performs no pool legitimacy check: it calls `transferFrom(swapper, pools[0], amount)` to move `tokenIn` to the first pool, then calls

`pool.getReserves()` and `pool.swap()` on every element of `pools` in sequence. A strategist can supply an all-malicious `pools` array where the first pool absorbs `tokenIn` and the last pool pre-funds the router with just enough `tokenOut` to pass oracle validation.

The `uniswapV3SwapTo` function in the same adapter routes through OpenOcean's `UniswapV3Exchange`, whose `uniswapV3SwapCallback` re-derives the expected pool address via `CREATE2` from the hardcoded Uniswap V3 factory and reverts if `msg.sender` does not match. A malicious pool cannot forge that callback origin, so the UniV3 path is not exploitable in the same way.

Other cases worth mentioning

`OneInch` has a `swap` function that sends arbitrary data to an `executor` address, configured by Veda. Similarly, OpenOcean's `swap`, `simpleSwap` and `swapGmxV2` functions make arbitrary calls to a `caller` address, also configured by Veda. The hops passed in to these functions should be verified to make sure there are no fake pools and the same attack depicted above cannot be executed here.

Impact

In all cases the post-execution oracle check in `_swapPostFlightCheck` is the only backstop. If `skipPriceValidation` is `true` for the route, the transaction succeeds with `tokenBalanceDelta = 0` and the full `tokenIn` amount is stolen. If oracle validation is active the attacker must supply enough `tokenOut` to pass the slippage check, limiting profit to the configured slippage tolerance.

Proof of concept

You can add the following test in `test/integrations/OneInchV6Integration.t.sol`, it shows as an example how the `unoswap` call does not validate the pool passed in:

```
...
// Simulates a malicious UniV2 pool: returns plausible token0/token1 and reserves
// to pass
// adapter checks, but absorbs tokenIn in swap() without sending any tokenOut.
contract MaliciousUniV2Pool {
    address public token0;
    address public token1;

    constructor(address _token0, address _token1) {
```

```
    token0 = _token0;
    token1 = _token1;
}

// Returns 0x60 bytes (required by _unoswapV2 returndatasize check)
function getReserves() external pure returns (uint112, uint112, uint32) {
    return (1_000_000e6, 1_000_000e6, 0);
}

// Called by 1inch router after it transfers tokenIn here; does nothing →
tokenIn stays
function swap(uint256, uint256, address, bytes calldata) external {}
}
...
function testV6UnoswapMaliciousPoolDrainsVault() external {
    _setUpMainnet();

    address usdt = getAddress(sourceChain, "USDT");
    address usdc = getAddress(sourceChain, "USDC");
    uint256 swapAmount = 1_000e6;
    deal(usdt, address(boringVault), swapAmount);

    MaliciousUniV2Pool maliciousPool = new MaliciousUniV2Pool(usdt, usdc);

    address[] memory dexes = new address[](1);
    dexes[0] = address(maliciousPool);

    ManageLeaf[] memory leafs = new ManageLeaf[](16);
    _addLeafsFor1InchV6Unoswap(leafs, usdt, dexes);

    bytes32[][] memory manageTree = _generateMerkleTree(leafs);
    manager.setManageRoot(address(this), manageTree[manageTree.length -
1][0]);

    Tx memory tx_ = _getTxArrays(2);

    tx_.manageLeafs[0] = leafs[0]; // approve USDT
    tx_.manageLeafs[1] = leafs[1]; // unoswap with malicious pool
```

```
bytes32[][] memory manageProofs = _getProofsUsingTree(tx_.manageLeafs,
manageTree);

tx_.targets[0] = usdt;
tx_.targets[1] = getAddress(sourceChain, "aggregationRouterV6");

tx_.targetData[0] = abi.encodeWithSignature(
    "approve(address,uint256)", getAddress(sourceChain,
"aggregationRouterV6"), type(uint256).max
);

// dex encodes maliciousPool address only (bits 253-255 = 0 → UniV2
protocol).
// numerator field (bits 160-191) = 0 → router uses default 997_000_000.
// minReturn = 0 → router's ReturnAmountIsNotEnough check is skipped
regardless of output.
tx_.targetData[1] = abi.encodeWithSignature(
    "unoswap(uint256,uint256,uint256,uint256)",
    uint256(uint160(usdt)),
    swapAmount,
    0, // minReturn = 0
    uint256(uint160(address(maliciousPool)))
);

tx_.decodersAndSanitizers[0] = rawDataDecoderAndSanitizer;
tx_.decodersAndSanitizers[1] = rawDataDecoderAndSanitizer;

// Does NOT revert: malicious pool is in the Merkle leaf and satisfies
the router's
// getReserves() + swap() interface, so the call completes successfully.
_submitManagerCall(manageProofs, tx_);

// Full tokenIn balance drained from boringVault
assertEq(ERC20(usdt).balanceOf(address(boringVault)), 0, "vault USDT not
drained");
// tokenIn absorbed by malicious pool – never returned as tokenOut
assertEq(ERC20(usdt).balanceOf(address(maliciousPool)), swapAmount, "pool
did not absorb tokenIn");
// boringVault received zero tokenOut
```

```
        assertEq(ERC20(usdc).balanceOf(address(boringVault)), 0, "unexpected  
tokenOut received");  
    }  
    ...
```

Recommendations:

For `OneInchAdapter.unoswap*` and `OpenOceanAdapter.callUniswap*`, validate pool addresses against a factory allowlist before accepting them. For UniV2-style pools, verify `IUniswapV2Factory(factory).getPair(tokenIn, tokenOut) == pool`; for UniV3-style pools, verify `IUniswapV3Factory(factory).getPool(tokenIn, tokenOut, fee) == pool`. The 1inch router also accepts a third protocol variant (Curve), the corresponding pool address should likewise be validated against a registry of known Curve pools or a Curve address provider before the adapter approves the call. Accepted factories and registries should be stored in the adapter and updatable only by governance.

For `OpenOceanAdapter.swap`, `simpleSwap`, and `swapGmxV2`, the adapter must inspect and validate the `calls` array rather than ignoring it. At minimum, each `CallDescription.target` should be restricted to a set of known legitimate pool contracts or factory-verified addresses.

Customer Response:

Fixed in [a87ccbd](#).

Fix Review:

Pools are all validated but we noticed there is no check in curve/UniV2/UniV3 that tokenIn is the first token input of the first pool. This is especially dangerous for UniV3 in OneInch. We created a new issue in the new report for this.

Medium Severity Issues

M-01: Strategist can submit a partial-fill 1inch limit order, breaking the protocol's full-fill invariant

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/adapters /OneInchAdapter.sol#L254-L288	Status: Acknowledged	

Description:

The protocol aims to reject partially-fillable limit orders.

While every adapter has such a check, the 1inch adapter doesn't.

`verifyLimitOrder()` decodes `OneInchLimitOrder` and validates assets, maker, and receiver, but never inspects `makerTraits`.

[1inch](#)'s `MakerTraitsLib` exposes:

- `NO_PARTIAL_FILLS_FLAG` at bit 255: by default this flag is unset, so an order routed through the 1inch limit-order protocol is partially fillable unless the maker **opts out**.
- `ALLOW_MULTIPLE_FILLS_FLAG` at bit 254: by default this flag is unset, so an order routed through the 1inch limit-order protocol is fillable only once unless the maker **opts in**.

This means the adapter should verify these two flags to only allow **one full-amount fill only**.

Allowing partial-fills in the current state of the protocol will produce undesirable side-effects.

Recommendations:

Reject any 1inch limit order that allows partial fills or multiple fills so only one full-amount fill is accepted.

Encode the high bits of `makerTraits` like such:

- **NO_PARTIAL_FILLS_FLAG** at bit 255 must be set to prevent partial fills from being accepted
- **ALLOW_MULTIPLE_FILLS_FLAG** at bit 254 must be unset (default) to prevent multiple fills from being accepted

```
uint256 private constant _NO_PARTIAL_FILLS_FLAG      = 1 << 255;
uint256 private constant _ALLOW_MULTIPLE_FILLS_FLAG = 1 << 254;
...
if ((order.makerTraits & _NO_PARTIAL_FILLS_FLAG) == 0) revert
OneInchAdapter__PartialFillsNotAllowed();
if ((order.makerTraits & _ALLOW_MULTIPLE_FILLS_FLAG) != 0) revert
OneInchAdapter__MultipleFillsNotAllowed();
```

Customer Response:

Acknowledged. Partial fills are now allowed, this is accepted behavior in that model.

M-02: `FeeRegistry.getFee()` returns the swapper default fee for pairs that were explicitly configured to zero

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/FeeRegistry.sol#L130-L143	Status: Fixed	

Description:

`getFee()` resolves the applicable fee using the group pair first, then falls back to `defaultFee`:

```
function getFee(address swapper, address tokenIn, address tokenOut) external view
returns (uint16) {
    uint8 groupIdIn = tokenGroup[swapper][tokenIn];
    uint8 groupIdOut = tokenGroup[swapper][tokenOut];
    uint16 feeBps = groupFees[swapper][_pairId(groupIdIn, groupIdOut)];

    //return fee for pair
    if (feeBps > 0) return feeBps;

    //if nothing set, return the default (which can be 0)
    feeBps = defaultFee[swapper];
    return feeBps;
}
```

The fallback fires for two distinguishable cases that share the same storage value (`groupFees[...] == 0`):

1. **Pair was never configured.** Falling back to the default is intended.
2. **Pair was explicitly set to 0 by an admin.** The admin's intent was "no fee for this pair", but the function silently ignores the configured zero and applies the default.

Because `setGroupPairFee` writes the literal value without a sentinel, there is no way for an operator to express "this specific pair is free" once a non-zero `defaultFee` exists.

Recommendations:

Consider adding a “flag” value to dissociate “no-fee” from “uninitialized fee” in `FeeRegistry::getFee()` and document the behavior.

For instance:- if `feeBps == type(uint16).max` consider it as “no-fee” and return 0.- if `feeBps == 0` consider it as “uninitialized fee” and return the `defaultFee`.

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed.

M-03: Strategist can refund the protocol fee on an executed limit order to the vault via miscalculated `availablePrincipal` in `_cancelOrder()`

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/BoringSwapper.sol#L265	Status: Fixed	

Description:

`_cancelOrder()` decides how much principal to refund using:

```
uint256 balance = record.tokenIn.balanceOf(address(this));
uint256 availablePrincipal = balance > record.fee ? balance - record.fee : 0;
uint256 refund = availablePrincipal < record.inputAmount ? availablePrincipal :
record.inputAmount;
```

The check subtracts only this order's own fee from the contract-wide balance. It never compares the remaining balance against the full locked floor (`pendingOrderPrincipal + feesInToken + claimableFees`), so it cannot detect that the order's principal has already left the swapper.

When the order has settled, the relayer has pulled `inputAmount` from the swapper: `balance` collapses to roughly `record.fee`, `availablePrincipal` collapses to zero, `refund` is zero, and the cancel proceeds without reverting.

The protocol's only remaining path for the order's fee is `releaseCancelFee()`, which transfers `record.fee` to `record.receiver` (the vault), not to the protocol's fee recipient.

A misbehaving strategist holding racing `cancelOrder()` against the protocol bot's `releaseFee()` call deterministically redirects every filled-order fee away from the treasury.

Recommendations:

Reject any cancel attempt that would violate the locked-floor invariant.

If `balanceOf(self, tokenIn) < pendingOrderPrincipal[T] + feesInToken[T] + claimableFees[T]`, some order in that token has settled off-chain and `releaseFee` has not yet run. Cancelling in that state is exactly what allows the redirect, so block it:

```
function _cancelOrder(uint256 orderId, SwapConfig calldata swapConfig) internal {
    OrderRecord storage record = orderRecords[orderId];
    if (address(record.tokenIn) == address(0)) revert
BoringSwapper__OrderNotFound();
    if (record.cancelledAt > 0) revert BoringSwapper__AlreadyCancelled();

    uint256 balance = record.tokenIn.balanceOf(address(this));
    uint256 expectedFloor = pendingOrderPrincipal[record.tokenIn]
        + feesInToken[record.tokenIn]
        + claimableFees[record.tokenIn];
    if (balance < expectedFloor) revert BoringSwapper__FloorViolation();

    // ... existing body
}
```

Declare `error BoringSwapper__FloorViolation();` alongside the other custom errors. When the strategist tries to cancel a filled order, `balance` is short by `record.inputAmount` and the guard reverts. The off-chain bot's `releaseFee` is the only path that can run, decrementing `pendingOrderPrincipal` and `feesInToken` and routing the fee into `claimableFees`. The protocol receives the fee on every fill.

Pair this with replacing the broken `availablePrincipal` formula so the refund block stops conflating contract-wide balance with per-order availability:

```
uint256 otherLocked =
    (pendingOrderPrincipal[record.tokenIn] - record.inputAmount)
    + feesInToken[record.tokenIn]
    + claimableFees[record.tokenIn];
uint256 availablePrincipal = balance > otherLocked ? balance - otherLocked : 0;
uint256 refund = availablePrincipal < record.inputAmount ? availablePrincipal :
record.inputAmount;
```

Operational note.

If two orders A and B share a token and A fills, an attempt to cancel B reverts until the bot calls `releaseFee(A)`. The coupling is bounded by the bot's responsiveness on fills and no funds are stuck (`releaseFee` remains callable).

If the project wants to eliminate the coupling, replace the floor guard with a per-order `filledAmount(swapConfig)` adapter view that proxies to `GPv2Settlement.filledAmount(orderUid)`; this gives a per-order signal independent of sibling state, at the cost of an `IAdapter` change.

Customer Response:

Fixed in [5c110ef](#).

Fix Review:

Fix confirmed.

M-04: Lack of check on filled orders allows premature fee release and already executed order cancellation

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/BoringSwapper.sol#L444-L464	Status: Fixed	

Description:

`releaseFee()` moves a limit-order's fee from the locked `feesInToken` bucket into the claimable `claimableFees` bucket. The natspec describes it as called "by the off-chain bot after confirming the fill", but the function itself does not verify on-chain that the order actually filled:

```
function releaseFee(uint256 orderId) external requiresAuth {
    OrderRecord memory record = orderRecords[orderId];
    if (address(record.tokenIn) == address(0)) revert
    BoringSwapper__OrderNotFound();
    if (record.cancelledAt > 0) revert BoringSwapper__AlreadyCancelled();

    delete orderRecords[orderId];
    approvedHashes[record.protocolHash] = false;
    ...
    feesInToken[record.tokenIn] -= record.fee;
    claimableFees[record.tokenIn] += record.fee;
    ...
}
```

The only checks are "order exists" and "order is not cancelled". An admin (or the compromised off-chain bot) can call `releaseFee(orderId)` the moment an order is submitted, before any fill happens. The fee then flows into `claimableFees` and becomes immediately collectable via `claimFees(token)`.

Setting `approvedHashes[record.protocolHash] = false` does invalidate the order from filling thereafter (since `isValidSignature` reverts on un-approved hashes). The principal stays in the

swapper but is no longer tracked in `pendingOrderPrincipal`, so `sweep()` will return it to the vault. The fee, however, is now claimable as if the trade actually happened.

Scenario

1. Strategist calls `submitOrder` for an order swapping 1_000_000 USDC at `feeBps = 50` (0.5%). `_limitOrderPreFlightCheck` computes `limitFee = 5_000 USDC`, transfers 1_005_000 USDC from the vault into the swapper, and increments `feesInToken[USDC] += 5_000`, `pendingOrderPrincipal[USDC] += 1_000_000`.
2. Before any taker fills the order, the admin/bot calls `releaseFee(orderId)`.
3. `feesInToken[USDC]` drops by 5_000; `claimableFees[USDC]` rises by 5_000; the order record is deleted; `approvedHashes[hash] = false`; `pendingOrderPrincipal[USDC]` drops by 1_000_000.
4. Admin calls `claimFees(USDC)`. 5_000 USDC is transferred to the configured fee recipient.
5. The order never fills (CoW now rejects the signature). Principal sits in the swapper, untracked, and is returned to the vault by `sweep(USDC)`.

Net: the vault paid 5_000 USDC in fees for a trade that never executed.

Additional impact

This oversight also allows a misbehaving strategist to execute `cancelOrder()` on a limit order that has already been filled by an external solver. Doing so moves the order's principal back to vault, reduces the swapper balance and prevents a pending order for the same token from being filled due to an insufficient token balance. The pending order would therefore need to be cancelled for the system to recover its optimal state.

Recommendations:

In each limit-order adapter, query the underlying protocol for the order's fill status during cancellation and fee release, then revert if the order has already been filled.

The bookkeeping update should only happen for genuinely-pending orders.

- **CoW (Settlement** `0x9008D19f58AAbD9eD0D60971565AA8510560ab41`): read `filledAmount(orderUid)` and revert if non-zero.

```
uint256 filled = IGPv2Settlement(cowSettlement).filledAmount(orderUid);
if (filled != 0) revert CowswapAdapter__OrderAlreadyFilled();
```

- **linch (router 0x11111125421cA6dc452d289314280a0f8842A65)**: if only full-amount fills are allowed (by setting `NO_PARTIAL_FILLS_FLAG` and leaving `ALLOW_MULTIPLE_FILLS_FLAG` unset as recommended in M-01), every order routed through the swapper uses linch's bit invalidator. The bit is 0 while the order is live and 1 after any fill, cancel.

Read that bit and revert if it is set:

```
uint256 nonce = (order.makerTraits >> 120) & ((1 << 40) - 1);
uint256 slot = nonce >> 8; // 256 nonces packed per storage slot
uint256 bitPos = nonce & 0xff; // position of this order's bit inside the slot

uint256 mask = IOneInchLOP(router).bitInvalidatorForOrder(order.maker, slot);
if (((mask >> bitPos) & 1) == 1) revert OneInchAdapter__OrderAlreadyFilled();
```

- **OpenOcean**: not strictly required since the cancellation reverts automatically, but extra-cautious would be to call `remainingRaw(orderHash)` on the order mixin and revert if non-zero (signals any partial fill).

Customer Response:

Fixed in [9decfdf](#) but applied only on the `cancelOrder()` flow.

Fix Review:

Fix confirmed. The issue still applies to the `releaseFee()` flow but it's assumed that the fee bot won't misbehave and will only execute this function when a limit order has genuinely been filled.

M-05: Instant swaps overwrite and clear token allowances of pending limit orders

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/BoringSwapper.sol#L552	Status: Fixed	

Description:

During an instant swap flow (`_swapPostFlightCheck`), the swapper explicitly approves the token to the target router, executes the swap, and immediately revokes the remaining allowance by setting it to 0.

If a live limit order utilizes the same approval target, the instant swap overrides and eventually nullifies the limit order's accumulated allowance, silently bricking off-chain fulfillment.

Moreover, the current implementation fails to first set the allowance to 0 before approving the desired swap amount which is necessary for USDT on Mainnet.

Recommendations:

Snapshot the existing allowance, set the allowance to 0 to handle USDT edge-case then set the new allowance to the amount for the atomic swap before performing the swap. After the atomic swap has been executed, restore the snapshotted allowance following the USDT edge-case precaution again. This handles the edge-case where a UniswapV3 swap doesn't consume the maximum input.

```
uint256 existingAllowance =
swapConfig.tokenRoute.tokenIn.allowance(address(this), target); // Snapshot

swapConfig.tokenRoute.tokenIn.safeApprove(target, 0); // USDT edge-case
swapConfig.tokenRoute.tokenIn.safeApprove(target, amount); // Approve atomic swap

(bool success,) = target.call(swapConfig.swapData); // Perform atomic swap
if (!success) revert BoringSwapper__SwapFailed();

// Restore pre-existing allowance (limit orders)
```

```
swapConfig.tokenRoute.tokenIn.safeApprove(target, 0);  
if (existingAllowance > 0) {  
    swapConfig.tokenRoute.tokenIn.safeApprove(target, existingAllowance);  
}
```

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

The fix is confirmed, but it introduced a new issue, which is documented in the follow-up audit report.

M-06: Slippage check in atomic swaps should be done with amounts after fees

Severity: Medium	Impact: Medium	Likelihood: High
Files: /src/base/Periphery/BoringSwapper.sol	Status: Acknowledged	

Description:

In `_swapPostFlightCheck`, the price and slippage check is performed against `tokenBalanceDelta` (the raw amount of `tokenOut` received from the external swap) before the protocol fee is deducted:

```
...
uint256 tokenBalanceDelta =
swapConfig.tokenRoute.tokenOut.balanceOf(address(this)) - tokenBalanceBefore;

//validate the price & slippage
IPriceValidator(priceValidator)
    .validate(
        swapConfig.tokenRoute.tokenIn,
        swapConfig.tokenRoute.tokenOut,
        amount,
        tokenBalanceDelta,
        swapConfig.quoteAsset,
        swapConfig.slippageBps
    );
...
```

And we can see later on how the `tokenBalanceDelta` includes the fee.

```
...
        uint256 fee = tokenBalanceDelta.mulDivDown(feeBps, 10_000);
        tokenBalanceDelta -= fee;
        swapConfig.tokenRoute.tokenOut.safeTransfer(feeRecipient, fee);
    }
}
```

```
    swapConfig.tokenRoute.tokenOut.safeTransfer(address(swapConfig.receiver),
tokenBalanceDelta);
...
```

The validation at `PriceValidator.validate()` requires:

$$\text{value}(\text{tokenBalanceDelta}) \geq \text{value}(\text{amount}) \times (1 - \text{slippageBps} / 10_000)$$

This guarantee holds for the gross output. The vault, however, receives only `tokenBalanceDelta - fee`. The actual slippage the vault experiences is therefore larger than `slippageBps` by an amount proportional to `feeBps`.

This discrepancy does not exist for limit orders: there, the fee is charged in `tokenIn` as an explicit add-on (`inputAmount + limitFee`) that does not touch the validated exchange rate, so `outputAmount` received by the vault is exactly what the slippage check approved.

Recommendations:

Perform the slippage validation after computing the fee, against the net output the vault will actually receive:

```
//charge fees if fee collection is active for this swapper
if (feeRegistry.swapperActive(address(this))) {
    uint16 feeBps = feeRegistry.getFee(
        address(this), address(swapConfig.tokenRoute.tokenIn),
address(swapConfig.tokenRoute.tokenOut)
    );
    if (feeBps > 0) {
        address feeRecipient = feeRegistry.getFeeRecipient(address(this),
swapConfig.tokenRoute.tokenOut);
        if (feeRecipient == address(0)) revert
BoringSwapper__FeeRecipientNotSet();
        uint256 fee = tokenBalanceDelta.mulDivDown(feeBps, 10_000);
        tokenBalanceDelta -= fee;
    }
}

// validate against NET output
IPriceValidator(priceValidator).validate(
```

```
        swapConfig.tokenRoute.tokenIn,  
        swapConfig.tokenRoute.tokenOut,  
        amount,  
        tokenBalanceDelta,  
        swapConfig.quoteAsset,  
        swapConfig.slippageBps  
    );  
  
    if (fee > 0) swapConfig.tokenRoute.tokenOut.safeTransfer(feeRecipient, fee);  
    swapConfig.tokenRoute.tokenOut.safeTransfer(address(swapConfig.receiver),  
tokenBalanceDelta);
```

This ensures the slippage tolerance configured by the strategist applies to the amount the vault actually receives.

Customer Response:

Acknowledged. The slippage here and the minimum output might be conflated. They are different variables. Slippage is on the trade itself, but is no guarantee of actual output of the trade. Using a fee as part of the slippage might be awkward for strategists pricing things.

M-07: `GenericRateProviderWithStalenessCheck` stores decimal metadata but does not normalize the rate

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: /src/helper/GenericRateProviderWithStalenessCheck.sol	Status: Fixed	

Description:

Function `PriceValidator._getPrices` computes the quote-asset value of a token amount as:

```
uint256 decimals = token.decimals();
uint256 idx;
for (uint256 i; i < rateProviders.length;) {
    uint256 primaryValue;
    {
        uint256 rate = IRateProvider(rateProviders[i]).getRate();
        if (rate == 0) revert PriceValidator__ZeroOracleRate();
        primaryValue = amount.mulDivDown(rate, 10 ** decimals);
    }
    ...
}
```

And for the intermediary hop:

```
uint256 baseRate = IRateProvider(baseRateProvider).getRate();
if (baseRate == 0) revert PriceValidator__ZeroOracleRate();
values[idx] = primaryValue.mulDivDown(baseRate, 1e18);
```

Both pieces of code assume that `getRate()` returns a value scaled to 18 decimal places, i.e., the price of one full token expressed as a fixed-point number with 18 digits of precision. Mixing rates at different scales across `tokenIn` and `tokenOut` breaks the comparison in `validate`.

If we check `GenericRateProviderWithDecimalScaling`, the contract correctly handles non-18-decimal sources:

```
function getRate() public override view returns (uint256) {
    uint256 rate = super.getRate();
    if (inputDecimals > outputDecimals) {
        return rate / 10 ** (inputDecimals - outputDecimals);
    } else if (inputDecimals < outputDecimals) {
        return rate * 10 ** (outputDecimals - inputDecimals);
    } else {
        return rate;
    }
}
```

But `GenericRateProviderWithStalenessCheck`, however, stores the same `inputDecimals` and `outputDecimals` fields but it never uses them. It overrides `getRate()` from `GenericRateProvider`, returning the raw oracle value verbatim:

```
...
    if (signed) {
        //if target func() returns an int, we get the result and then cast it
to a uint256
        int256 res = abi.decode(result, (int256));
        if (res < 0) revert GenericRateProvider__PriceCannotBeLtZero();

        return uint256(res);

    } else {

        return abi.decode(result, (uint256));
    }
}
```

As a result, non-18-decimal rates can cause the `PriceValidator.validate()` to either revert because the slippage restriction will never be met or bypass the slippage check completely.

Recommendations:

Implement decimal normalization in `GenericRateProviderWithStalenessCheck.getRate()`, mirroring what `GenericRateProviderWithDecimalScaling` already does:

```
function getRate() public override view returns (uint256) {
    ...
```

```
+    uint256 rate;
    if (signed) {
        //if target func() returns an int, we get the result and then cast it
to a uint256
        int256 res = abi.decode(result, (int256));
        if (res < 0) revert GenericRateProvider__PriceCannotBeLtZero();

-        return uint256(res);
+        rate = uint256(res);

    } else {

-        return abi.decode(result, (uint256));
+        rate = abi.decode(result, (uint256));
    }
+    if (inputDecimals > outputDecimals) return rate / 10 ** (inputDecimals -
outputDecimals);
+    if (inputDecimals < outputDecimals) return rate * 10 ** (outputDecimals -
inputDecimals);
+    return rate;
}
```

Customer Response:

Fixed in [a87ccbd](#).

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Protocol fee should round up, in favor of the protocol

Severity: Low	Impact: Low	Likelihood: Low
Files: BoringSwapper.sol#L620-L629	Status: Fixed	

Description:

`_limitOrderPreFlightCheck()` computes the limit-order fee with `mulDivDown`:

```
limitFee = info.inputAmount.mulDivDown(feeBps, 10_000);  
feesInToken[swapConfig.tokenRoute.tokenIn] += limitFee;
```

Rounding down on the fee biases every truncation in favor of the trader and against the protocol. The convention in fee accounting is the opposite: round the fee up so the protocol never loses dust and the trader never gains it.

The instant-swap path in `_swapPostFlightCheck()` has the same issue.

Recommendations:

Use `mulDivUp` for fee computation in both the limit-order and instant-swap paths

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed.

L-02: Slippage threshold rounds down, letting swaps with slightly worse output pass

Severity: Low	Impact: Low	Likelihood: Low
Files: PriceValidator.sol#L42-L49	Status: Fixed	

Description:

`PriceValidator.validate()` derives the minimum acceptable output value as:

```
uint256 minValueOut = valuesIn[i].mulDivDown((10_000 - slippageBps), 10_000);  
if (valuesOut[j] < minValueOut) revert PriceValidator__ExceedsMaxSlippage();
```

`mulDivDown` truncates `minValueOut`, so the check accepts outputs that are 1 wei below the strict threshold.

The slippage check should be the stricter of the two interpretations: the protocol wants to reject as many bad swaps as possible, not accept them. Rounding the minimum up makes the threshold tighter and the comparison stricter.

Recommendations:

Use `mulDivUp` for the minimum-output computation:

```
uint256 minValueOut = valuesIn[i].mulDivUp((10_000 - slippageBps), 10_000);  
if (valuesOut[j] < minValueOut) revert PriceValidator__ExceedsMaxSlippage();
```

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

L-03: Lowering `maxFeeBps` does not retroactively cap existing fee configurations

Severity: Low	Impact: Low	Likelihood: Low
Files: /src/base/Periphery/FeeRegistry.sol	Status: Fixed	

Description:

`maxFeeBps` is intended to be a global ceiling on any fee that can be charged across all swapper instances. The cap is enforced at write time in `setGroupPairFee` and `setDefaultFee`:

```
function setGroupPairFee(address swapper, uint8 groupA, uint8 groupB, uint16
feeBps) external requiresAuth {
    if (feeBps > maxFeeBps) revert FeeRegistry__FeeTooHigh();
    groupFees[swapper][_pairId(groupA, groupB)] = feeBps;
    emit GroupPairFeeSet(swapper, groupA, groupB, feeBps);
}
...
function setDefaultFee(address swapper, uint16 feeBps) external requiresAuth
{
    if (feeBps > maxFeeBps) revert FeeRegistry__FeeTooHigh();
    defaultFee[swapper] = feeBps;
    emit DefaultFeeSet(swapper, feeBps);
}
```

However, `getFee` (the function called at swap time) reads the stored value without applying the cap:

```
function getFee(address swapper, address tokenIn, address tokenOut) external
view returns (uint16) {
    uint8 groupIdIn = tokenGroup[swapper][tokenIn];
    uint8 groupIdOut = tokenGroup[swapper][tokenOut];
    uint16 feeBps = groupFees[swapper][_pairId(groupIdIn, groupIdOut)];

    //return fee for pair
    if (feeBps > 0) return feeBps;
```

```
    //if nothing set, return the default (which can be 0)
    feeBps = defaultFee[swapper];
    return feeBps;
}
```

And `setMaxFeeBps` updates the cap without updating any previously stored fee:

```
function setMaxFeeBps(uint16 newMaxFeeBps) external requiresAuth {
    maxFeeBps = newMaxFeeBps;
    emit MaxFeeBpsUpdated(newMaxFeeBps);
}
```

It would be expected that lowering `maxFeeBps` immediately limits all active fees, but the existing entries are preserved until they are explicitly updated.

Recommendations:

Enforce the cap dynamically inside `getFee` so the effective fee is always bounded by the current `maxFeeBps`, regardless of when the stored value was written:

```
function getFee(address swapper, address tokenIn, address tokenOut) external
view returns (uint16) {
    uint8 groupIdIn = tokenGroup[swapper][tokenIn];
    uint8 groupIdOut = tokenGroup[swapper][tokenOut];
    uint16 feeBps = groupFees[swapper][_pairId(groupIdIn, groupIdOut)];

-    //return fee for pair
-    if (feeBps > 0) return feeBps;
-
-    //if nothing set, return the default (which can be 0)
-    feeBps = defaultFee[swapper];
-    return feeBps;
+    if (feeBps == 0) feeBps = defaultFee[swapper];
+    return feeBps > maxFeeBps ? maxFeeBps : feeBps;
}
```

Customer Response:



Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

L-04: `verifyLimitOrder` does not validate extension-to-salt binding, relying on 1inch protocol-level check

Severity: Low	Impact: Low	Likelihood: Low
Files: src/base/Periphery/adapters/ OneInchAdapter.sol#L254-L288	Status: Acknowledged	

Description:

1inch LOP v6 orders can carry an "extension", a bytes blob encoding post-interaction hooks like FeeTaker, which redirects filled tokens to a custom receiver.

The extension is not part of the EIP-712 order struct. Instead, 1inch binds it through the `salt` field: the lower 160 bits of `order.salt` must equal `uint160(keccak256(extension))`.

At fill time, 1inch's `OrderMixin._isValidExtension()` enforces this, meaning each order commits to exactly one extension at creation. The upper 96 bits of `salt` remain available for nonce/entropy. `OneInchAdapter.verifyLimitOrder()` decodes the extension and validates that the embedded receiver matches `swapConfig.receiver`, but never verifies that the submitted extension is the one committed to by `order.salt`:

```
(DecoderCustomTypes.OneInchLimitOrder memory order, bytes memory extension) =
    abi.decode(swapConfig.swapData, (DecoderCustomTypes.OneInchLimitOrder,
bytes));

// ... receiver checks

bytes memory orderData = abi.encode(order);
bytes32 orderHash = _computeOrderHash(orderData); // extension not included
```

The receiver check confirms the extension currently being submitted points to the vault.

It does not confirm this extension matches what `order.salt` commits to. The adapter silently depends on 1inch's own `_isValidExtension` to reject mismatched extensions at fill time.

Impact

No direct exploit against current linch Limit Order Protocol v6 since `_isValidExtension()` blocks extension substitution at fill time.

Recommendations:

The code can be hardened by verifying the order's salt matches the extension hash. Add the salt-extension binding check in `verifyLimitOrder()`, after the receiver validation:

```
if (extension.length > 0) {
    uint160 extHash = uint160(uint256(keccak256(extension)));
    if (extHash != uint160(order.salt)) revert
OneInchAdapter__ExtensionNotBoundToSalt();
}
```

This makes the adapter self-sufficient regardless of linch's checks.

Customer Response:

Acknowledged. `OneInchAggregationRouterV6` is not an upgradeable contract, and the checks performed in that contract are sufficient to be relied on for our purposes.

L-05: Rebasing tokens can break swap and order accounting

Severity: Low	Impact: Low	Likelihood: Low
Files: src/base/Periphery/BoringSwapper.sol#L595	Status: Acknowledged	

Description:

`_limitOrderPreFlightCheck()` pulls `info.inputAmount + limitFee` from the receiver and stores both as fixed numbers:

```
feesInToken[swapConfig.tokenRoute.tokenIn] += limitFee;
pendingOrderPrincipal[swapConfig.tokenRoute.tokenIn] += info.inputAmount;
swapConfig.tokenRoute.tokenIn
    .safeTransferFrom(address(swapConfig.receiver), address(this),
info.inputAmount + limitFee);
```

For rebasing tokens like Lido `stETH`, the swapper's real balance drifts while the order is pending.

Positive rebases (most days) grow the balance above `pendingOrderPrincipal + feesInToken`. Negative rebases (rare, triggered by validator penalties exceeding daily rewards) shrink it below.

`stETH` also has a known 1–2 wei rounding loss per `transferFrom()` due to share-to-balance conversion, so the swapper starts ~1 wei short even before any rebase. The cancel path caps the refund at `record.inputAmount` in one direction and at the current balance in the other:

```
uint256 availablePrincipal = balance > record.fee ? balance - record.fee : 0;
uint256 refund = availablePrincipal < record.inputAmount ? availablePrincipal :
record.inputAmount;
if (refund > 0) record.tokenIn.safeTransfer(address(record.receiver), refund);
```

Positive drift: `refund` is capped at `record.inputAmount` and the accrual stays stranded on the swapper.

Negative drift: `refund` falls below `record.inputAmount` and the receiver loses principal.

On the fill path, the limit protocol pulls exactly `info.inputAmount` from the swapper. Under a negative rebase that drops the swapper's balance below `inputAmount + fee`, making the pull reverts mid-settlement and the order is un-fillable until balance recovers.

Recommendations:

Consider not integrating rebase tokens.

Customer Response:

Acknowledged. Positive rebases are recoverable by the vault via `sweep()` and negative rebases are extremely unlikely but accepted. Tokens can be sent to top up the balance of the vault in extreme cases of a negative rebase.

L-06: A solver can tamper with ``quoteAsset`` and ``slippageBps`` at fill time to bypass the price check in ``isValidSignature()``

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/base/Periphery/BoringSwapper.sol#L302-L303	Status: Fixed	

Description:

`isValidSignature()` is the EIP-1271 callback limit order protocols invoke to re-validate a resting limit order at fill time. It decodes the entire `SwapConfig` from the solver-supplied `_signature` blob and forwards `quoteAsset` and `slippageBps` straight to `PriceValidator.validate()`:

```
SwapConfig memory swapConfig = abi.decode(_signature, (SwapConfig));
...
IPriceValidator(priceValidator).validate(
    ERC20(info.inputToken),
    ERC20(info.outputToken),
    info.inputAmount,
    info.outputAmount,
    swapConfig.quoteAsset,    // solver-supplied
    swapConfig.slippageBps    // solver-supplied
);
```

Neither parameter is bound to the order: `_computeOrderHash()` hashes only `swapData`.

`OrderRecord` does not store `quoteAsset` or `slippageBps` either. The `approvedHashes[_hash]` check therefore cannot constrain them.

This lets the solver substitute any `(quoteAsset, slippageBps)` pair in `_signature` while keeping `protocolHash` identical to the strategist's submission. The validator then runs with solver-chosen parameters.

Recommendations:

Store `quoteAsset` and `slippageBps` in `OrderRecord` at submission time.

The `isValidSignature()` function must read from storage and ignore the solver-supplied values.

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed.

L-07: Wrong path validation in `UniswapV3Adapter.exactInput` allows a malicious strategist to steal from the swapper

Severity: Low	Impact: Medium	Likelihood: Low
Files: /src/base/Periphery/adapters/UniswapV3Adapter.sol	Status: Fixed	

Description:

`UniswapV3Adapter.exactInput` extracts `pathTokenIn` and `pathTokenOut` by reading the first and last 20 bytes of the raw path without verifying that the path conforms to the canonical Uniswap V3 encoding:

```
// verify path tokens match the approved token route
// path is abi.encodePacked(tokenIn, fee, ..., tokenOut) – first 20 bytes =
tokenIn, last 20 bytes = tokenOut
bytes memory path = params.path;
address pathTokenIn;
address pathTokenOut;
assembly {
    pathTokenIn := shr(96, mload(add(path, 0x20)))
    pathTokenOut := shr(96, mload(add(add(path, 0x20), sub(mload(path),
20))))
}
if (ERC20(pathTokenIn) != swapConfig.tokenRoute.tokenIn) revert("path tokenIn
mismatch");
if (ERC20(pathTokenOut) != swapConfig.tokenRoute.tokenOut) revert("path
tokenOut mismatch");
```

A canonical Uniswap V3 multi-hop path has the structure:

```
token0 (20 bytes) | fee0 (3 bytes) | token1 (20 bytes) | fee1 (3 bytes) | ... |
tokenn (20 bytes)
```

Valid constraints: `path.length >= 43` and `(path.length - 20) % 23 == 0`.

However, the adapter does not verify that the `path` conforms to this structure. Looking at the actual `exactInput` function from Uniswap:

```
/// @inheritdoc ISwapRouter
function exactInput(ExactInputParams memory params)
    external
    payable
    override
    checkDeadline(params.deadline)
    returns (uint256 amountOut)
{
    address payer = msg.sender; // msg.sender pays for the first hop

    while (true) {
        bool hasMultiplePools = params.path.hasMultiplePools();

        // the outputs of prior swaps become the inputs to subsequent ones
        params.amountIn = exactInputInternal(
            params.amountIn,
            hasMultiplePools ? address(this) : params.recipient, // for
intermediate swaps, this contract custodies
            0,
            SwapCallbackData({
                path: params.path.getFirstPool(), // only the first pool in
the path is necessary
                payer: payer
            })
        );

        // decide whether to continue or terminate
        if (hasMultiplePools) {
            payer = address(this); // at this point, the caller has paid
            params.path = params.path.skipToken();
        } else {
            amountOut = params.amountIn;
            break;
        }
    }
}
```

```
require(amountOut >= params.amountOutMinimum, 'Too little received');  
}
```

with

```
/// @dev The length of the bytes encoded address  
uint256 private constant ADDR_SIZE = 20;  
/// @dev The length of the bytes encoded fee  
uint256 private constant FEE_SIZE = 3;  
  
/// @dev The offset of a single token address and pool fee  
uint256 private constant NEXT_OFFSET = ADDR_SIZE + FEE_SIZE;  
/// @dev The offset of an encoded pool key  
uint256 private constant POP_OFFSET = NEXT_OFFSET + ADDR_SIZE;  
/// @dev The minimum length of an encoding that contains 2 or more pools  
uint256 private constant MULTIPLE_POOLS_MIN_LENGTH = POP_OFFSET +  
NEXT_OFFSET;  
  
/// @notice Returns true iff the path contains two or more pools  
/// @param path The encoded swap path  
/// @return True if path contains two or more pools, otherwise false  
function hasMultiplePools(bytes memory path) internal pure returns (bool) {  
    return path.length >= MULTIPLE_POOLS_MIN_LENGTH;  
}
```

We can see that `exactInput` loops until the path no longer has multiple pools, i.e., until its length drops below 66 bytes. A strategist who mistakenly submits a path such as:

```
WETH (20 bytes) | fee (3 bytes) | WBTC (20 bytes) | fill (2 bytes) | USDC (20  
bytes)
```

would cause Uniswap to execute a WETH → WBTC swap, while the `SwapConfig` specifies a WETH → USDC route. The adapter's two token checks pass because the first and last 20 bytes of the 65-byte path still read as WETH and USDC respectively. The result is that the swapper's WBTC balance increases while USDC output is zero (`tokenBalanceDelta = 0`). For the transaction to succeed rather than revert at the slippage check, at least one of the `tokenIn` or `tokenOut` tokens must be configured to skip price validation. A different strategist could then notice the extra WBTC balance and execute a small swap that routes WBTC as `tokenIn`, receiving the balance back via the dust-return mechanism:

```
...
    // Return any unspent tokenIn dust to the vault, excluding locked limit order
    funds.
    uint256 locked = pendingOrderPrincipal[swapConfig.tokenRoute.tokenIn]
        + feesInToken[swapConfig.tokenRoute.tokenIn]
        + claimableFees[swapConfig.tokenRoute.tokenIn];
    uint256 balance = swapConfig.tokenRoute.tokenIn.balanceOf(address(this));
    uint256 dust = balance > locked ? balance - locked : 0;
    if (dust > 0)
swapConfig.tokenRoute.tokenIn.safeTransfer(address(swapConfig.receiver), dust);
...
```

Recommendations:

Validate the path length before extracting token addresses:

```
bytes memory path = params.path;
if (path.length < 43 || (path.length - 20) % 23 != 0) revert("invalid path
length");
```

This ensures the path encodes a whole number of (fee, token) hops and that the final 20 bytes the adapter reads are actually the last token in the decoded sequence — the same token the Uniswap V3 router will deliver output to.

Customer Response:

Fixed in [a87ccbd](#).

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Failed swap consumes additional gas before reverting

Files:

BoringSwapper.sol#L551-L553

Description:

In `_swapPostFlightCheck()`, the swap call resets the approval before checking the success flag:

```
(bool success,) = target.call(swapConfig.swapData);
swapConfig.tokenRoute.tokenIn.safeApprove(target, 0);
if (!success) revert BoringSwapper__SwapFailed();
```

When the external swap fails and the entire transaction is going to revert, `safeApprove` between the call and the revert burns gas only to undo work that the revert will discard.

Recommendations:

Reorder the two statements so the revert happens before the approval reset:

```
(bool success,) = target.call(swapConfig.swapData);
if (!success) revert BoringSwapper__SwapFailed();
swapConfig.tokenRoute.tokenIn.safeApprove(target, 0);
```

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed.

I-02: Refactor and document `BoringSwapper::replaceOrder()`

Files:

BoringSwapper.sol#L219-L232

Description:

`submitOrder()` and `replaceOrder()` end with the same three steps: pre-flight checks, rate limit consume, and emit `OrderSubmitted`. The replace path simply prefixes a `_cancelOrder()` call.

These three steps can be encapsulated into an internal function to reduce code size and increase readability.

Moreover, the `replaceOrder()` function intention is ambiguous due to a lack of natspec and could hint at creating a new order but using the same `orderId`.

Recommendations:

1. Extract the shared body into an internal helper and have both external functions call it:

```
function _submitOrder(SwapConfig memory swapConfig) internal {
    (bytes32 key, IAdapter.OrderInfo memory info, uint256 orderId) =
    _limitOrderPreFlightCheck(swapConfig);
    if (!_consumeRateLimit(key, swapConfig.tokenRoute.tokenIn, info.inputAmount))
    {
        revert BoringSwapper__RateLimitExceeded();
    }
    emit OrderSubmitted(orderId, key, info.inputAmount,
    address(swapConfig.receiver));
}
```

1. Add natspec to `replaceOrder()` stating its purpose (cancel existing order + create a brand new order).

Customer Response:

Fixed in [fb7d2bb](#).

Fix Review:

Fix confirmed.

I-03: `getFeeRecipient()` natspec describes a fee, not a recipient

Files:

FeeRegistry.sol#L145-L153

Description:

`getFeeRecipient()` returns a fee recipient address, but its natspec was copy-pasted from `getFee()` and still says it "returns the applicable fee for a swap pair".

Recommendations:

Replace the existing natspec to match the function's intention.

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

I-04: Unsanitized `swapData` selector allows limit-order adapter functions to be dispatched through the atomic swap path, producing misinterpreted return values

Files:

/src/base/Periphery/BoringSwapper.sol

Description:

`_swapPreFlightCheck` sends `swapConfig.swapData` as raw calldata to the adapter via `functionStaticCall`:

```
bytes memory appended =
    abi.encodePacked(swapConfig.swapData, abi.encode(swapConfig),
uint256(swapConfig.swapData.length));
(bytes memory result) = adapter.functionStaticCall(appended);
(target, amount) = abi.decode(result, (address, uint256));
```

The first four bytes of `swapData` are the function selector that gets dispatched. The code assumes these bytes will always identify a legitimate swap function (`swap`, `simpleSwap`, `exactInput`, etc.), but nothing enforces this. A strategist can set those bytes to the selector of any `view` function on the adapter, including functions intended solely for the limit-order path.

The result of the static call will be interpreted as `(target, amount)` by the swapper, but a static call to `verifyLimitOrder` or `cancelLimitOrder` on those adapters that implement limit orders will return different values, misleading the swapper.

For instance, in the case of `CowSwapAdapter`, `verifyLimitOrder` will return:

```
return OrderInfo({
    approvalTarget: vaultRelayer,
    cancelTarget: cowSettlement,
    inputToken: order.sellToken,
    outputToken: order.buyToken,
    inputAmount: order.sellAmount,
    outputAmount: order.buyAmount,
    protocolHash: orderHash
});
```

where `vaultRelayer` will be interpreted as `target` and `cowSettlement` as `amount`.

For `cancelLimitOrder`, the return is:

```
return (cowSettlement, abi.encodeWithSignature("invalidateOrder(bytes)",
orderUid));
```

so `target` will be `cowSettlement` and `amount` will be `abi.encodeWithSignature("invalidateOrder(bytes)", orderUid)`.

`amount` is then passed to `_swapPostFlightCheck` to try to transfer from the `receiver`:

```
swapConfig.tokenRoute.tokenIn.safeTransferFrom(address(swapConfig.receiver),
address(this), amount);
```

Since `amount` will be extremely large in the case of `verifyLimitOrder`, this always reverts. But for `cancelLimitOrder`, which returns `(address, bytes memory)`, the second ABI word is the byte offset of the dynamic data (`0x40 = 64`), so `amount = 64`. In this case `safeTransferFrom` would succeed (64 atomic units), an unintended approval for 64 units would be granted to `cowSettlement`, and `cowSettlement.call(swapData)` would be attempted. This will most likely revert, because the chances that the same selector of `cancelLimitOrder` also exists in the `cowSettlement` contract are very low (the selector has 32 bits, so this would be a collision between two random elements out of more than four billion possibilities). But in case it existed, it would allow an unintended call to that contract.

Recommendations:

Validate the function selector at the start of `swapData` against a per-adapter allowlist of permitted swap selectors before dispatching:

```
bytes4 selector = bytes4(swapConfig.swapData);
if (!IAdapter(swapConfig.adapter).isSwapSelector(selector)) revert
BoringSwapper__InvalidSwapSelector();
```

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

I-05: Missing upper-bound validation on basis-point values allows fees above 100%, which would cause DoS on atomic swaps

Files:

/src/base/Periphery/FeeRegistry.sol

Description:

Fee and slippage values throughout the protocol are expressed in basis points with a denominator of `10_000` (100 %). Neither `FeeRegistry` nor `BoringSwapper` enforces the invariant that these values must be at most `10_000`.

In `FeeRegistry.setMaxFeeBps` there is no range check:

```
function setMaxFeeBps(uint16 newMaxFeeBps) external requiresAuth {
    maxFeeBps = newMaxFeeBps; // @audit-info Sanitization: maxFeeBps should
    be within a valid range?
    emit MaxFeeBpsUpdated(newMaxFeeBps);
}
```

Because `setGroupPairFee` and `setDefaultFee` only check `feeBps > maxFeeBps`, setting `maxFeeBps` above `10_000` allows stored fees above 100% to pass validation.

An admin mistake that sets a fee above 100% would make the following snippet from `_swapPostFlightCheck` revert because of an underflow:

```
uint256 fee = tokenBalanceDelta.mulDivDown(feeBps, 10_000);
tokenBalanceDelta -= fee; // underflows when fee > tokenBalanceDelta
```

making impossible to swap through the swapper until the `feeBps` is set back to a reasonable value.

Recommendations:

Add an explicit upper-bound check in the `setMaxFeeBps` setter:

```
function setMaxFeeBps(uint16 newMaxFeeBps) external requiresAuth {
    if (newMaxFeeBps > 10_000) revert FeeRegistry__FeeTooHigh();
    maxFeeBps = newMaxFeeBps;
}
```

```
    emit MaxFeeBpsUpdated(newMaxFeeBps);  
}
```

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

I-06: Duplicate `protocolHash` is accepted

Files:

src/base/Periphery/BoringSwapper.sol#L651

Description:

If two orders produce the same hash (e.g. strategist resubmits identical CoW order fields, or **salt** collision on 1inch), both reference one shared **approvedHashes** slot.

Cancel/release on either revokes the shared flag, immediately bricking ERC-1271 fills for the other. Principal is locked, allowance is still set, but un-fillable until cancelled manually.

Recommendations:

Before setting **approvedHashes** to **true**, ensure it isn't already set and revert if that is the case.

Customer Response:

Fixed in [9decfdf](#).

Fix Review:

Fix confirmed.

I-07: `cancelOrder` / `replaceOrder` lack `nonReentrant`

Files:

src/base/Periphery/BoringSwapper.sol#L215

Description:

Neither of `cancelOrder()` and `replaceOrder()` has the `nonReentrant` modifier while `swap()` and `submitOrder()` both use it.

Recommendations:

Extra insurance can be added by applying the `nonReentrant` modifier on these functions.

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

I-08: `OneInchAdapter` does not block the WETH-unwrap flag on `unoswap*` paths and `fillOrder`

Files:

`/src/base/Periphery/adapters/OneInchAdapter.sol`

Description:

`OpenOceanAdapter` explicitly blocks all pool-level WETH flags that would cause the router to deliver raw ETH instead of an ERC-20 token. Both UniV2 and UniV3 direct-path helpers enforce this at the same point where `tokenOut` is derived:

```
// UniV2 path - _getUniV2DstToken
uint256 internal constant WETH_MASK = 1 << 254;
if (rawPool & WETH_MASK != 0) revert OpenOceanAdapter__WethFlagsNotAllowed();

// UniV3 output path - _getUniV3DstToken
uint256 internal constant WETH_UNWRAP_MASK = 1 << 253;
if (lastPool & WETH_UNWRAP_MASK != 0) revert
OpenOceanAdapter__WethFlagsNotAllowed();
```

`OneInchAdapter` applies no equivalent guard. In the `linch V6 Address` library, bit 252 of any `dex` parameter signals that the router should unwrap WETH to ETH before delivering the output:

```
uint256 private constant _WETH_UNWRAP_FLAG = 1 << 252;
...
function shouldUnwrapWeth(Address self) internal pure returns(bool) {
    return self.getFlag(_WETH_UNWRAP_FLAG);
}
```

All six `unoswap` variants resolve `tokenOut` through `_getTokenOutUni`, which reads `token0()/token1()` from `address(uint160(dex))`. Since `_WETH_UNWRAP_FLAG` sits at bit 252, above the 160-bit pool address, it is silently ignored by the pool extraction. The adapter therefore reports `tokenOut = WETH` for a route that the `linch` router will actually settle in ETH:

```
function _getTokenOutUni(uint256 dex, address tokenIn) internal view returns
(address) {
    address pool = address(uint160(dex)); // bit 252 is discarded here
```

```
        return IUniswapV3(pool).token0() == tokenIn ? IUniswapV3(pool).token1() :
IUniswapV3(pool).token0();
    }
```

This affects every function that reaches `_getTokenOutUni`: `unoswap`, `unoswap2`, `unoswap3`, `unoswapTo`, `unoswapTo2`, and `unoswapTo3`.

`fillOrder` has a parallel gap. The linch V6 `TakerTraitsLib` exposes an `unwrapWeth` flag at bit 254 of `takerTraits`. When set, the router unwraps WETH to ETH before delivering the maker asset to the taker. The current guard checks only `_ARGS_HAS_TARGET` (bit 251):

```
uint256 private constant _ARGS_HAS_TARGET = 1 << 251;

function fillOrder(..., uint256 takerTraits) external view returns (address,
uint256) {
    ...
    if (takerTraits & _ARGS_HAS_TARGET != 0) revert
OneInchAdapter__CustomTargetNotAllowed();
    // bit 254 (_UNWRAP_WETH) is not checked
    return (router, amount);
}
```

Because `BoringSwapper` has no `receive()` function, the practical consequence in both cases is a transaction revert (`BoringSwapper__SwapFailed`) rather than fund loss: the linch router attempts to push ETH to the swapper, the transfer fails, and the entire call rolls back. However, the adapter's `tokenOut` prediction is wrong whenever these flags are set, and the absence of a guard is inconsistent with the explicit protection already present in `OpenOceanAdapter`.

Recommendations:

Add the WETH-unwrap check to `_getTokenOutUni` so that all six `unoswap` variants are covered in one place, mirroring the pattern used in `OpenOceanAdapter`:

```
uint256 private constant _WETH_UNWRAP_FLAG = 1 << 252;

function _getTokenOutUni(uint256 dex, address tokenIn) internal view returns
(address) {
    if (dex & _WETH_UNWRAP_FLAG != 0) revert
OneInchAdapter__WethUnwrapNotAllowed();
}
```

```
        address pool = address(uint160(dex));
        return IUniswapV3(pool).token0() == tokenIn ? IUniswapV3(pool).token1() :
IUniswapV3(pool).token0();
    }
}
```

Add the corresponding guard to **fillOrder**:

```
uint256 private constant _UNWRAP_WETH = 1 << 254;

function fillOrder(..., uint256 takerTraits) external view returns (address,
uint256) {
    ...
    if (takerTraits & _ARGS_HAS_TARGET != 0) revert
OneInchAdapter__CustomTargetNotAllowed();
    if (takerTraits & _UNWRAP_WETH != 0) revert
OneInchAdapter__WethUnwrapNotAllowed();
    return (router, amount);
}
```

Customer Response:

Fixed in [49b0000](#).

Fix Review:

Fix confirmed.

I-09: Malicious strategist can route vault funds to arbitrary contracts via uninspected inner call targets in LI.FI and OpenOcean adapters

Files:

src/base/Periphery/adapters/LifiAdapter.sol#L39-L100
src/base/Periphery/adapters/OpenOceanAdapter.sol#L92-L141

Description:

Both [LifiAdapter](#) and [OpenOceanAdapter](#) validate token routes, receivers, and amounts, but pass inner call targets through to the aggregator without inspection.

In [LifiAdapter](#), the [LifiSwapData](#) struct contains [callTo](#), [approveTo](#), and [callData](#). None of these fields are read by the adapter:

```
function swapTokensGeneric(
    ...
    DecoderCustomTypes.LifiSwapData[] calldata _swapData
) external view returns (address, uint256) {
    if (_receiver != payable(msg.sender)) revert
    LifiAdapter__ReceiverNotSwapper();
    // checks sendingAssetId, receivingAssetId only
    // callTo, approveTo, callData remain UNVALIDATED
    return (router, _swapData[0].fromAmount);
}
```

In [OpenOceanAdapter](#), the [OpenOceanCallDescription\[\] calls](#) parameter is entirely ignored in [swap\(\)](#), [simpleSwap\(\)](#), and [swapGmxV2\(\)](#):

```
function swap(
    address caller,
    DecoderCustomTypes.OpenOceanSwapDescription calldata desc,
    DecoderCustomTypes.OpenOceanCallDescription[] calldata /*calls*/ // ignored
) external view returns (address, uint256) {
```

Each [OpenOceanCallDescription](#) contains an arbitrary [target](#) (address packed as [uint256](#)), [value](#), and [data](#). The aggregator router executes these calls during settlement. The adapter provides no filtering.

This isn't an exploitable issue at the moment. Two external checks currently mitigate this:

- the aggregator's own balance-delta enforcement (`balanceAfter - balanceBefore >= minAmount`)
- the swapper's `PriceValidator` oracle check in `_swapPostFlightCheck`

Recommendations:

Consider hardening the adapters by whitelisting allowed inner call targets at the adapter level. Both adapters should validate that every inner target is a known DEX contract before returning.

For `LifiAdapter`, validate each `LifiSwapData` entry:

```
mapping(address => bool) public allowedCallTargets;

function _validateSwapData(DecoderCustomTypes.LifiSwapData calldata data)
internal view {
    if (!allowedCallTargets[data.callTo]) revert LifiAdapter__CallToNotAllowed();
    if (data.approveTo != data.callTo && !allowedCallTargets[data.approveTo])
        revert LifiAdapter__ApproveToNotAllowed();
}
```

For `OpenOceanAdapter`, stop ignoring the `calls` parameter and validate each entry's target:

```
mapping(address => bool) public allowedCallTargets;

function _validateCalls(DecoderCustomTypes.OpenOceanCallDescription[] calldata
calls) internal view {
    for (uint256 i; i < calls.length;) {
        address target = address(uint160(calls[i].target));
        if (!allowedCallTargets[target]) revert
OpenOceanAdapter__CallTargetNotAllowed();
        unchecked { i++; }
    }
}
```

Admin maintains the whitelist of approved DEX contracts.

Customer Response:

Acknowledged. It is not feasible to whitelist every possible route an aggregator might integrate. This is the purpose of slippage checks, to check the deltas of the trade. What happens in the interim matters much less.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.